

# Synthetic Data Generation with Generative AI

Text, Image, Audio & Tabular — Additional Lecture 2 of 3

---

Ezequiel López-Rubio

Friday, September 25, 2026

Malmö University, Sweden

DA631E — Artificial Intelligence for Data Science

## Recap and today's step

Last time: **augmentation** — *transform* existing labelled examples (text, image, audio, tabular).

### Today: synthetic data generation

Create entirely new, artificial examples — sometimes with no seed data at all — in **all four modalities**.

#### Augmentation

real example → variant of it

#### Synthetic data

description / schema → new example

# Outline

1. Why synthetic data?
2. The landscape
3. Generative techniques
4. Evaluating & trusting synthetic data
5. Practical in Google Colab
6. Wrap-up

## Why synthetic data?

---

# Why generate data that isn't real?

- **Privacy:** model behaviour without exposing real people (GDPR is a first-class concern in the EU and Sweden).
- **Scarcity:** too little data for a new or niche problem.
- **Labelling cost:** generate examples *with* labels directly.
- **Rare events / edge cases:** fraud, faults, unusual sounds or images.
- **Class imbalance:** manufacture minority-class examples.
- **Cold start:** bootstrap a prototype before real data exists.

# Synthetic data is not magic

## The honest caveat

Synthetic data reflects only what the generator “knows”. It can **miss real patterns**, **inherit biases**, and **invent plausible-but-wrong** examples.

So everything builds toward one question:

*Is the synthetic data **good enough**?*

We answer it informally today; **formally in Lecture 7**.

# The landscape

---

# Generators by modality

| Modality | Classic methods            | Generative-AI method (today)            |
|----------|----------------------------|-----------------------------------------|
| Text     | templates, grammars        | <b>LLM</b> (instruction-prompted)       |
| Tabular  | SMOTE, Bayesian nets, GANs | <b>LLM</b> → structured JSON            |
| Image    | GANs, VAEs                 | <b>diffusion</b> (few-step, text→image) |
| Audio    | concatenative synthesis    | <b>neural TTS</b> (text→speech)         |

All four share one idea: *condition a generator on a specification* and sample new, labelled examples.

GANs = Generative Adversarial Networks, VAEs = Variational Autoencoders



# Where generative models shine (and don't)

## Strong fit

- Natural language & labelled NLP data.
- Small structured records with semantics.
- Plausible images / speech for prototyping.

## Weaker fit

- Precise numeric correlations.
- High-fidelity, safety-critical realism.
- Guarantees on statistical validity.

In practice, generative and classic methods are often **combined**.

# Generative techniques

---

## Core techniques (modality-agnostic)

- **Zero-shot:** describe what you want; sample it.
- **Few-shot:** give 2–3 examples to fix format and style.
- **Conditioning for diversity:** vary *attributes* — personas, classes, scenes, speakers.
- **Structured output:** request **JSON** for tabular data.
- **Direct labelling:** the label is known *by construction*.
- **Distillation / self-instruct:** one model generates training data for another.

# Diversity must be engineered

A naive prompt (“write a review”, “a photo of a cat”) yields **repetitive** output.

## Fixes (all modalities)

- **Condition** on attributes (product, tone; breed, lighting; speaker, word).
- Iterate over a **grid** of attribute values.
- Raise the sampling **temperature** / **strength** (watch quality).
- **De-duplicate** afterwards.

*Diversity does not appear by itself — you must design for it.*

# Structured generation for tabular data

Ask for **JSON** conforming to a schema, then **validate and parse**:

- Define fields, types, and allowed ranges in the prompt.
- Request one JSON object (or list) **and nothing else**.
- **Never trust the output**: parse defensively, drop malformed rows.

## Validation is part of the method

Small models often emit *almost*-valid JSON. Robust parsing and schema checks are not optional.

# Evaluating & trusting synthetic data

---

# Three questions for any synthetic dataset

1. **Fidelity**: does it *look* real? (distributions, plausibility)
2. **Utility**: is it *useful* for the task?
3. **Privacy**: does it *leak* real individuals?

## TSTR — the key utility test

Train on **S**ynthetic, Test on **R**real. Good performance on *real* held-out data means the synthetic data carried real signal.

Proper methodology + metrics: Lecture 7. Today: a first taste.

## Risks specific to synthetic data

- **Bias amplification:** the generator's biases become your dataset's.
- **Privacy leakage / memorisation:** models can echo training data.
- **Factual / physical errors:** confident but wrong records or artefacts.
- **Model collapse:** training repeatedly on synthetic output degrades quality.
- **Governance:** licensing, provenance, documentation.

### Document it

Record *how* data was made (model, prompts, date): a “datasheet” for your synthetic set.



# Practical in Google Colab

---

## Practical: plan and constraints

**Goal.** Generate a small *labelled* dataset in *each* modality and inspect it.

### Ground rules (free-Colab friendly)

- Runs entirely in **Google Colab**; enable the free **T4 GPU**.
- **Tens** of items per modality, not thousands.
- Load one generator at a time; **free memory** between sections.

Models: Qwen2.5-0.5B-Instruct (text/tabular), sd-turbo (image), SpeechT5 (audio).

## Practical: memory helper (run once)

```
import gc, torch
def free_memory(*objs):
    for o in objs:
        try: del o
        except: pass
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
print("GPU available:", torch.cuda.is_available())
```

Same discipline as Lecture 1: one generator at a time keeps us inside the free tier.

## Modality 1 — Text: labelled examples from an LLM

```
!pip -q install "transformers>=4.44" accelerate diffusers datasets \
    soundfile sentencepiece scikit-learn

from transformers import pipeline
gen = pipeline("text-generation", model="Qwen/Qwen2.5-0.5B-Instruct",
               torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
               device_map="auto")

def chat(user, system="You generate concise training data.", temp=0.9, max_new=80):
    msgs = [{"role": "system", "content": system}, {"role": "user", "content": user}]
    return gen(msgs, max_new_tokens=max_new, do_sample=True,
               temperature=temp, top_p=0.95)[0]["generated_text"][-1]["content"].strip()

print(chat("Write one short, casual POSITIVE review of wireless earbuds."))
```

One model serves the text *and* tabular sections; we free it once, afterwards.

## Text: an attribute grid builds diversity

```
import itertools, random, pandas as pd
products    = ["earbuds", "coffee maker", "running shoes", "laptop stand"]
sentiments  = ["positive", "negative"]
tones       = ["casual", "detailed", "annoyed", "enthusiastic"]

grid = list(itertools.product(products, sentiments, tones)); random.shuffle(grid)
rows = []
for prod, sent, tone in grid[:24]:                # keep the workload small
    txt = chat(f"Write ONE {sent} review of a {prod}. Tone: {tone}. "
               f"1-2 sentences. Return only the review.", temp=1.0)
    rows.append({"text": txt, "label": sent, "product": prod, "tone": tone})

df_text = pd.DataFrame(rows).drop_duplicates("text").reset_index(drop=True)
print(df_text.shape); df_text.head()
```

The label is known **by construction** — no manual labelling.

## Modality 2 — Tabular: structured JSON + validation

```
import json
def gen_record():
    p = ('Return ONE JSON object only, no extra text: ',
        '{"age": 18-80, "plan": "basic|pro|enterprise", ',
        '"tenure_months": 0-72, "tickets": 0-15, "churned": true|false}'' )
    raw = chat(p, system="You output strict JSON only.", max_new=90, temp=0.8)
    try:
        rec = json.loads(raw[raw.find("{"): raw.rfind("}")+1])
        assert 18 <= rec["age"] <= 80                # schema check
        return rec
    except Exception:
        return None                                  # drop malformed output

data = [r for r in (gen_record() for _ in range(40)) if r][:30]
df_tab = pd.DataFrame(data); print("kept", len(df_tab), "valid rows"); df_tab.head()
free_memory(gen)    # release the text model before the image model
```

Reuses the chat model from the text section — loaded once, freed here.

## Modality 3 — Image: text→image with diffusion

```
from diffusers import AutoPipelineForText2Image
t2i = AutoPipelineForText2Image.from_pretrained(
    "stabilityai/sd-turbo",
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32)
t2i = t2i.to("cuda" if torch.cuda.is_available() else "cpu")

classes = {"apple": "a photo of a single red apple on a white background",
          "banana": "a photo of a single yellow banana on a white background"}
images = []
for label, prompt in classes.items():
    for i in range(3):
        img = t2i(prompt, num_inference_steps=1, guidance_scale=0.0).images[0]
        images.append((img, label))
print("generated", len(images), "labelled images"); images[0][0]
free_memory(t2i)      # release the image model before the audio model
```

sd-turbo needs **one** step — a labelled image set in seconds.

## Modality 4 — Audio: text→speech (TTS)

```
from transformers import pipeline
from datasets import load_dataset
import soundfile as sf

tts = pipeline("text-to-speech", model="microsoft/speecht5_tts")
emb = load_dataset("Matthijs/cmu-arctic-xvectors", split="validation")
speaker = torch.tensor(emb[7306]["xvector"]).unsqueeze(0)    # a fixed voice

words, clips = ["yes", "no", "up", "down"], []
for w in words:
    out = tts(w, forward_params={"speaker_embeddings": speaker})
    sf.write(f"{w}.wav", out["audio"], out["sampling_rate"])
    clips.append((f"{w}.wav", w))                            # (file, label)
print("wrote", len(clips), "labelled clips"); free_memory(tts)    # release the audio model
```

A tiny spoken-keyword set. Vary speaker & words to grow it.



## A first “peek”: TSTR on the text data

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

real = [("These earbuds sound amazing and fit perfectly.", "positive"),
        ("The coffee maker leaked and died in a week.", "negative"),
        ("Super comfortable shoes for long runs.", "positive"),
        ("The laptop stand is flimsy and wobbles.", "negative")]
Xr, yr = zip(*real)

clf = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000))
clf.fit(df_text["text"], df_text["label"])      # TRAIN on synthetic
print("TSTR accuracy on real:", clf.score(Xr, yr))
```

**Caveat:** four real points is only a demo. Proper TSTR + metrics = Lecture 7.

## Practical: what to observe and discuss

- **Text**: how repetitive is output *without* the attribute grid?
- **Tabular**: how many JSON rows were invalid? Any impossible values?
- **Image**: are the classes visually distinct? Any weird artefacts?
- **Audio**: are the spoken words intelligible? How diverse across speakers?
- Which **biases** appear, and where might they come from?

## Wrap-up

---

## Key takeaways

1. Synthetic data **creates** new labelled examples in every modality.
2. One recipe: *condition a generator on a specification, then sample*.
3. **Diversity is engineered**; **validate** structured output; **document** provenance.
4. Watch bias, privacy leakage, and **model collapse**.
5. Judge by **fidelity, utility (TSTR), privacy** — formalised in Lecture 7.

### Additional Lecture 3 — Hands-on Workshop

You drive. Combine augmentation and synthetic generation across modalities, and **evaluate properly** using the tools from Lecture 7.

Questions?

## Further reading (optional)

- Wang et al. (2023), *Self-Instruct*.
- Long et al. (2024), *On LLMs-Driven Synthetic Data Generation: A Survey*.
- Borisov et al. (2023), *Language Models are Realistic Tabular Data Generators*.
- Rombach et al. (2022), *Latent Diffusion*; Sauer et al. (2023), *Adversarial Diffusion Distillation (SD-Turbo)*.
- Ao et al. (2022), *SpeechT5*; Shumailov et al. (2024), *Model Collapse*.